

FYP Proposed Project Statement (Revised Version)

A. Project Background

Large Language Models (LLMs) have fundamentally transformed modern artificial intelligence, enabling breakthroughs in natural language understanding, programming assistance, and complex multi-step reasoning. However, as these models are deployed in real-world applications — such as analyzing long legal documents, processing entire codebases, or maintaining extended conversational context — they encounter a severe hardware bottleneck commonly referred to as the “Memory Wall.”

The Attention Mechanism and Its Quadratic Cost

The architectural core of every modern LLM is the Scaled Dot-Product Attention mechanism. Standard attention requires the model to compute pairwise similarity scores between every token and every other token in the input sequence. Formally, given input matrices $Q, K, V \in \mathbb{R}^{(N \times d)}$, the computation is:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V$$

The intermediate matrix $S = QK^T \in \mathbb{R}^{(N \times N)}$ is the root of the problem. As sequence length N grows, the memory required to store this matrix grows quadratically — $O(N^2)$. To provide a concrete illustration: at sequence length $N = 16,384$ tokens with 32-bit floats, the attention matrix alone requires approximately 4 GB of GPU memory. Doubling the input sequence length quadruples the memory consumption, making long-context inference and training practically infeasible on consumer and even mid-range professional hardware.

A Survey of Prior Efficient Attention Methods

The research community has pursued two broad strategies to address this quadratic memory bottleneck, each with distinct trade-offs.

1. Algorithmic Approximations (Accuracy-Memory Trade-offs)

Early work attempted to reduce the quadratic cost by computing only a subset of attention scores:

- Sparse Attention (Child et al., 2019) introduced strided and local attention patterns, restricting each token to attend only to a fixed window of neighbors and a small set of global tokens. While this reduced complexity to $O(N\sqrt{N})$, the enforced sparsity patterns were hand-designed and failed to capture all relevant long-range dependencies, resulting in measurable accuracy degradation on tasks that require global context.
- Longformer (Beltagy et al., 2020) combined sliding-window local attention with task-specific global attention tokens. This achieved $O(N)$ complexity and demonstrated strong performance on document-level tasks, but the global token selection required task-specific engineering and did not generalise as a drop-in replacement for full attention.
- Linformer (Wang et al., 2020) approximated the $N \times N$ attention matrix by projecting the Key and Value matrices to a lower-dimensional representation of fixed rank k , yielding $O(N)$ complexity. However, the low-rank approximation assumption breaks down for tasks where attention distributions are highly non-uniform, and the authors acknowledged accuracy losses on tasks requiring precise token-level attention.
- Performer (Choromanski et al., 2021) used random feature approximations to estimate the softmax kernel without materialising the attention matrix, achieving $O(N)$ time and space complexity. Despite its theoretical elegance, Performer suffered from unstable approximation quality in practice and required careful tuning of the number of random features.

The common limitation of all approximation-based approaches is that they sacrifice exact attention semantics — they compute a fundamentally different function than standard attention, making them unsuitable for settings where mathematical correctness is required, including model training where gradient accuracy is critical.

2. Hardware-Aware Exact Attention (IO-Aware Algorithms)

A fundamentally different strategy was introduced by Dao et al. (2022), who observed that the bottleneck in standard attention is not the number of floating-point operations, but rather the memory bandwidth cost of repeatedly reading and writing the large intermediate attention matrix between the GPU's slow High-Bandwidth Memory (HBM) and its fast on-chip cache (SRAM).

FlashAttention (Dao et al., 2022) reformulated the attention computation as a tiled algorithm that processes data in small blocks that fit entirely within SRAM, completely eliminating the need to materialize the full $N \times N$ matrix in HBM. By combining this tiling strategy with a numerically stable Online Softmax technique, FlashAttention computes exact attention — mathematically identical to standard attention — while achieving $O(N)$ memory complexity and 2–4× wall-clock training speedup on GPT-style models. Crucially, unlike all approximation-based prior work, FlashAttention introduces zero accuracy degradation in either the forward or backward pass.

FlashAttention-2 (Dao, 2023) refined the original algorithm with improved work partitioning across GPU thread blocks, reducing non-matmul FLOPs and better exploiting the asymmetry between the B_r and B_c block dimensions. FlashAttention-2 reported a further 2× speed than FlashAttention-1, achieving up to 72% of theoretical GPU peak FLOP/s on A100 GPUs. The backward pass in FlashAttention-2 was also significantly redesigned: rather than storing the $N \times N$ attention matrix during backpropagation, it uses rematerialization — recomputing the attention scores tile by tile during the backward pass from the saved $O(N)$ statistics, trading additional compute for dramatically reduced memory.

FlashAttention-3 (Shah et al., 2024) extended the algorithm to exploit NVIDIA Hopper architecture features, including asynchronous memory pipelines and FP8 low-precision computation, pushing peak utilization further on H100-class hardware which mainly Nvidia focus.

PyTorch FlexAttention represents a complementary approach: rather than providing a single fused kernel, It exposes a compiler-driven interface that allows users to define custom attention score modifications as Python functions, which are automatically compiled into fused GPU kernels via `torch.compile`. This offers flexibility that the original FlashAttention implementation lacked, at the cost of additional compilation overhead.

Why This Project Is Significant

The progression from approximate methods to IO-aware exact algorithms represents one of the most impactful recent developments in practical AI systems engineering. This project implements both the forward and backward passes of FlashAttention from first principles in OpenAI's Triton framework — producing a complete, training-capable implementation that can be used for actual model fine-tuning, not merely inference. Understanding and implementing both passes bridges the critical gap between high-level machine learning mathematics and low-level GPU memory hierarchy optimization, and produces a reproducible artifact for studying hardware-aware algorithm design at its full depth.

B. Problem / Improvement Areas

The Core Engineering Problem

In standard deep learning frameworks, computing attention forces the GPU to execute a multi-phase memory access pattern:

- Forward pass: Compute $S = QK^T$, write $N \times N$ matrix to HBM; read S back, compute softmax, write result to HBM; read again to compute weighted sum with V
- Backward pass: Read the stored $N \times N$ attention matrix back from HBM to compute gradients dQ , dK , dV

This generates $O(N^2)$ HBM read/write operations in both the forward and backward passes, entirely bandwidth-bound — the GPU's thousands of compute cores sit idle waiting for data to arrive from memory. On an A100 GPU, HBM bandwidth is approximately 2 TB/s while SRAM bandwidth is roughly 19 TB/s — a $\sim 10\times$ gap that FlashAttention directly exploits. During training, the backward pass memory bottleneck is even more severe than the forward pass because the full $N \times N$ attention matrix must be retained in HBM between the forward and backward passes for gradient computation, doubling the sustained memory pressure.

Engineering Goals and Hypotheses

This project addresses the following explicitly defined objectives:

Goal	Description	Success Criterion
Forward Memory Elimination	Eliminate $N \times N$ matrix storage in HBM during forward pass	Peak VRAM ≤ 2 GB at $N = 16,384$ on RTX 3090
Backward Memory Elimination	Eliminate stored $N \times N$ matrix for backward pass via recomputation	Backward VRAM matches forward VRAM — $O(N)$ scaling confirmed
Linear Memory Scaling	Achieve $O(N)$ memory for complete forward + backward pass	Empirically confirm linear growth from $N = 1K$ to $16K$
Forward Correctness	Output mathematically identical to standard attention	<code>torch.allclose(output, reference, rtol=1e-3, atol=1e-3)</code> passes
Backward Correctness	Gradients mathematically identical to standard attention backward	<code>torch.autograd.gradcheck()</code> passes on small inputs; gradient comparison passes on large inputs
Comparative Benchmarking	Quantify trade-offs against three production baselines	Throughput (TFLOPs/s) and VRAM measured across all implementations for both passes
Training Validation	Demonstrate complete training capability	Loss decreases over iterations on a small GPT-2 style transformer block using custom kernels
Implementation Clarity	Produce readable, documented Triton kernels	Forward kernel ≤ 250 lines, backward kernel ≤ 350 lines, both with docstring annotations

Quantitative Hypothesis — Forward Pass: At sequence length $N = 16,384$, standard PyTorch attention will require approximately 4 GB of VRAM for the attention matrix alone ($N^2 \times 4$ bytes in FP32). The tiled forward pass will reduce peak VRAM to approximately $O(N \times d)$ — under 500 MB total — a reduction of 8× or greater.

Quantitative Hypothesis — Backward Pass: Standard attention backward requires storing the full $N \times N$ attention matrix between forward and backward passes, adding another ~4 GB at $N = 16,384$. The recomputation-based backward pass will eliminate this entirely, maintaining the same $O(N)$ memory footprint as the forward pass, at the cost of approximately 2× additional FLOPs compared to the forward pass alone.

Scope Decision — Forward Pass First, Backward Pass Gated: The backward pass will be implemented as a gated Phase 2, conditional on the forward pass being fully implemented and benchmarked by end of December 2026. If this gate is not met, the backward pass will be documented mathematically as future work with a detailed technical analysis. This gate is explicitly agreed upon with the project supervisor.

C. Proposed Approach

1. Core Algorithm: FlashAttention Forward Pass

1.1 Tiling (Matrix Blocking)

The input matrices $Q \in \mathbb{R}^{(N \times d)}$, $K \in \mathbb{R}^{(N \times d)}$, and $V \in \mathbb{R}^{(N \times d)}$ are divided into blocks of size $B_r \times d$ (for Q) and $B_c \times d$ (for K and V), where B_r and B_c are chosen such that a block of Q , a block of K , and a block of V fit simultaneously into SRAM:

$$B_r \cdot d + 2 \cdot B_c \cdot d \leq M_{SRAM}$$

where M_{SRAM} is the available on-chip shared memory capacity of the target GPU (typically 96 KB to 228 KB depending on architecture). For $d = 64$ and an RTX 3090 with 99 KB shared memory per SM, this yields $B_r = B_c = 64$ or 128 as starting candidates, subject to empirical autotuning.

1.2 Online Softmax Algorithm

Standard softmax requires two passes over a complete row — incompatible with tiled processing where the full row is never available at once. The Online Softmax algorithm (Milakov & Gimelshein, 2018) maintains running statistics corrected as each new block is processed.

Algorithm: Online Softmax Tiled Attention Forward Pass

Require: $Q \in \mathbb{R}^{(N \times d)}$, $K \in \mathbb{R}^{(N \times d)}$, $V \in \mathbb{R}^{(N \times d)}$

Block sizes B_r, B_c ; scale factor $T = 1/\sqrt{d}$

Initialize for each row-block i of Q :

$O_i \leftarrow 0 \in \mathbb{R}^{(B_r \times d)}$ # Output accumulator

$L_i \leftarrow 0 \in \mathbb{R}^{(B_r)}$ # Running normalizer (sum of exponentials)

$m_i \leftarrow -\infty \in \mathbb{R}^{(B_r)}$ # Running row maximum

For each column-block $j = 1$ to T_c (where $T_c = N / B_c$):

Load K_j, V_j from HBM $\rightarrow$ SRAM

For each row-block $i = 1$ to T_r (where $T_r = N / B_r$):

Load Q_i from HBM $\rightarrow$ SRAM

Load current O_i, L_i, m_i from HBM $\rightarrow$ SRAM

Step 1: Compute raw attention scores for this tile

$S_{ij} \leftarrow T \cdot Q_i \cdot K_j^T$ # Shape: $B_r \times B_c$

Step 2: Apply causal mask

$S_{ij}[r, c] \leftarrow -\infty$ for all $c > \text{global_col_index}(j, r)$

Step 3: Compute new local row maximum

$m_{ij} \leftarrow \text{rowmax}(S_{ij})$ # Shape: B_r

Step 4: Update global running maximum

$m_{i_new} \leftarrow \max(m_i, m_{ij})$ # Element-wise

Step 5: Compute correction factors

$\alpha \leftarrow \exp(m_i - m_{i_new})$ # Rescale old statistics

$\beta \leftarrow \exp(m_{ij} - m_{i_new})$ # Rescale new block

Step 6: Compute locally shifted exponentials

$P_{ij} \leftarrow \exp(S_{ij} - m_{i_new}[:, \text{None}])$ # Shape: $B_r \times B_c$

Step 7: Update running normalizer

$l_{i_new} \leftarrow \alpha * l_i + \text{rowsum}(P_{ij})$

Step 8: Update output accumulator

$O_{i_new} \leftarrow \alpha * O_i + P_{ij} \cdot V_j$ # Shape: $B_r \times d$

Step 9: Store updated statistics

$m_i \leftarrow m_{i_new}$

$l_i \leftarrow l_{i_new}$

$O_i \leftarrow O_{i_new}$

After all column blocks: normalize

$O_i \leftarrow O_i / l_i[:, \text{None}]$

Save for backward pass

Save $m \in \mathbb{R}^N$, $l \in \mathbb{R}^N$ to HBM

Write $O \rightarrow \text{HBM}$

The mathematical correctness guarantee is that after all T_c column-blocks have been processed, the accumulated output O_i equals exactly the standard softmax-weighted sum. The saved statistics m (row maxima) and l (normalizers) are the only additional storage required by the backward pass — $O(N)$ rather than $O(N^2)$.

1.3 Causal Masking

The causal mask is integrated inside the tiled loop rather than as a separate preprocessing step. When processing tile (i, j) , any position where the global column index exceeds the global row index receives a score of $-\infty$ before the exponential is applied, contributing zero weight to the softmax output without requiring a separate mask matrix in HBM.

1.4 Triton Forward Kernel

The forward pass kernel will be implemented in OpenAI's Triton framework, where each program instance handles one (batch, head, row-block) tuple. Triton abstracts register allocation and memory coalescing while exposing explicit block-level parallelism, making the implementation significantly more maintainable than raw CUDA C++.

2. Core Algorithm: FlashAttention Backward Pass

The backward pass is the more algorithmically complex component. It must compute three gradients — dQ , dK , and dV — without storing the $N \times N$ attention matrix, using the saved $O(N)$ statistics (m , l) and recomputation.

2.1 Mathematical Derivation

Given the loss gradient $dO \in \mathbb{R}^{(N \times d)}$ from downstream computation, the three required gradients are derived as follows.

Gradient with respect to V :

$$dV = P^T \cdot dO$$

where $P = \text{softmax}(S)$ is the attention probability matrix. This is computed tile by tile by recomputing P_{ij} from the saved m and l statistics.

Gradient with respect to the attention scores S:

The softmax backward formula for a row vector p with upstream gradient dp is:

$$ds_i = p_i \left(dp_i - \sum_j dp_j p_j \right) = p_i (dp_i - D_i)$$

where the per-row scalar $D_i = \text{rowsum}(dO_i \odot O_i)$ must be precomputed before the main backward tiling loop. This precomputation is implemented as a separate lightweight kernel.

Gradient with respect to Q and K:

$$dQ_i = \sum_j dS_{ij} \cdot K_j, \quad dK_j = \sum_i dS_{ij}^T \cdot Q_i$$

2.2 Recomputation Strategy

Rather than storing the full P matrix from the forward pass, the backward pass recomputes P_{ij} on the fly for each tile using the saved m_i and l_i :

$$P_{ij} = \exp(T \cdot Q_i \cdot K_j^T - m_i[:, \text{None}]) / l_i[:, \text{None}]$$

This recomputation is the key mechanism that maintains $O(N)$ memory during the backward pass at the cost of approximately $2\times$ additional FLOPs compared to storing P.

2.3 Full Backward Pass Algorithm

Causal masking requires no special reproducibility mechanism between the forward and backward passes. In contrast to dropout, which requires explicit RNG state preservation to ensure the backward pass recomputation matches the forward pass exactly.

Dropout support is treated as a conditional stretch-goal extension, gated behind full completion and validation of the core forward and backward passes without dropout. Unlike causal masking, dropout cannot be implemented in the forward pass alone — if applied, the identical dropout mask must be reproduced during backward pass recomputation to compute correct gradients, since the backward pass never sees the actual mask used, only the recomputed tile.

This reproducibility requirement will be addressed using a Philox counter-based random number generator, consistent with FlashAttention-2's approach: rather than storing an $O(N^2)$ dropout mask, a seed and per-tile offset are saved during the forward pass, allowing the identical mask to be regenerated deterministically during backward recomputation. This extension will only be attempted after the no-dropout forward and backward passes are fully validated (see Gate Condition, Section D), and will be documented as future work if not reached within the project timeline.

Algorithm: FlashAttention Backward Pass

Require: $Q, K, V \in \mathbb{R}^{(N \times d)}$ — original inputs

$O \in \mathbb{R}^{(N \times d)}$ — forward pass output

$dO \in \mathbb{R}^{(N \times d)}$ — upstream gradient

$m \in \mathbb{R}^N, l \in \mathbb{R}^N$ — saved forward statistics

Block sizes B_r, B_c ; scale factor $T = 1/\sqrt{d}$

Phase 1: Precompute per-row D scalars (separate lightweight kernel)

For each row i :

$D_i \leftarrow \text{rowsum}(dO_i \odot O_i)$ # Shape: N (scalar per row)

Phase 2: Initialize gradient accumulators

$dQ \leftarrow 0 \in \mathbb{R}^{(N \times d)}$

$dK \leftarrow 0 \in \mathbb{R}^{(N \times d)}$

$dV \leftarrow 0 \in \mathbb{R}^{(N \times d)}$

Phase 3: Main tiled backward loop

For each column-block $j = 1$ to T_c :

Load K_j, V_j from HBM $\rightarrow$ SRAM

Initialize local $dK_j, dV_j \leftarrow 0$

For each row-block $i = 1$ to T_r :

Load Q_i, O_i, dO_i from HBM $\rightarrow$ SRAM

Load m_i, l_i, D_i from HBM $\rightarrow$ SRAM

Step 1: Recompute attention scores (no stored matrix needed)

$S_{ij} \leftarrow T \cdot Q_i \cdot K_j^T$ # Shape: $B_r \times B_c$

Step 2: Apply causal mask (identical to forward)

$S_{ij}[r, c] \leftarrow -\infty$ for all $c > \text{global_col_index}(j, r)$

Step 3: Recompute attention probabilities using saved statistics

$P_{ij} \leftarrow \exp(S_{ij} - m_i[:, \text{None}]) / \ell_i[:, \text{None}]$

Step 4: Gradient w.r.t. V (no atomic needed — j loop is outer)

$dV_j \leftarrow dV_j + P_{ij}^T \cdot dO_i$ # Shape: $B_c \times d$

Step 5: Gradient w.r.t. P

$dP_{ij} \leftarrow dO_i \cdot V_j^T$ # Shape: $B_r \times B_c$

Step 6: Gradient w.r.t. S using softmax backward formula

$dS_{ij} \leftarrow P_{ij} \odot (dP_{ij} - D_i[:, \text{None}])$ # Shape: $B_r \times B_c$

Step 7: Gradient w.r.t. K (accumulated locally, no atomic)

$dK_j \leftarrow dK_j + dS_{ij}^T \cdot Q_i$ # Shape: $B_c \times d$

Step 8: Gradient w.r.t. Q (requires atomic add — multiple j blocks

write to same Q_i rows)

$dQ_i \leftarrow dQ_i + dS_{ij} \cdot K_j$ # Shape: $B_r \times d$

`atomic_add(dQ[i], dQ_i)`

Write completed dK_j, dV_j blocks to HBM

Write $dK_j, dV_j \rightarrow \text{HBM}$

Write $dQ \rightarrow \text{HBM}$

2.4 The Atomic Addition Problem for dQ

The most technically challenging aspect of the backward pass is that computing dQ_i requires contributions from all T_c column-blocks of K . In a parallelized Triton kernel where different program instances handle different (i, j) combinations, multiple instances attempt to write to the same dQ rows simultaneously. Three strategies will be evaluated:

Strategy	Mechanism	Trade-off
Atomic additions	<code>tl.atomic_add()</code> for dQ writes	Correct but slower due to memory contention
Sequential j -loop	Each program handles all j for a fixed i	Avoids atomics, limits parallelism
Separate reduction kernel	Accumulate partial dQ per j , then reduce	Adds kernel launch overhead

The sequential j -loop approach (used in FlashAttention-2's backward design) will be the primary implementation strategy, with atomic addition as a validated fallback.

Performance implications of each strategy will be measured and reported.

2.5 PyTorch Autograd Integration

The complete forward and backward kernels will be integrated into PyTorch's autograd system via `torch.autograd.Function`:

```
class FlashAttentionFunction(torch.autograd.Function):
```

```
    @staticmethod
```

```
    def forward(ctx, Q, K, V, causal):
```

```
        O, m, l = flash_forward_kernel(Q, K, V, causal)
```

```
        ctx.save_for_backward(Q, K, V, O, m, l)
```

```
ctx.causal = causal
```

```
return O
```

```
@staticmethod
```

```
def backward(ctx, dO):
```

```
    Q, K, V, O, m, l = ctx.saved_tensors
```

```
    dQ, dK, dV = flash_backward_kernel(Q, K, V, O, dO, m, l,  
                                         ctx.causal)
```

```
    return dQ, dK, dV, None
```

This integration enables the custom implementation to be used as a drop-in replacement within any PyTorch training loop.

3. Core Design Philosophy

Principle 1 — Kernel Fusion

Rather than decomposing attention into a pipeline of separate GPU kernel launches (score computation, softmax normalization, weighted aggregation), the entire forward pass is implemented as a single fused kernel. This eliminates the $O(N^2)$ HBM round-trips that occur when intermediate results are written to and read back from main memory between separate kernels. The fused kernel keeps all intermediate tile computations within SRAM from the moment Q_i , K_j , and V_j are loaded until the output O_i is written — a single HBM write per row-block rather than three separate $O(N^2)$ read-write cycles.

Principle 2 — Block-Local Accumulation

Both the forward and backward passes assign each GPU thread block ownership of a fixed row-block index i . That thread block iterates over all column blocks j sequentially, accumulating results locally in registers and SRAM. This design eliminates the need for atomic memory operations in the critical accumulation paths. Specifically:

- Forward pass: O_i , L_i , m_i accumulated locally $\rightarrow$ single normalized write at end
- Backward pass: dK_j and dV_j accumulated locally within the column-block loop $\rightarrow$ no atomics
- Backward pass: dQ_i accumulated locally across all j iterations $\rightarrow$ single write per row-block

The only remaining atomic contention risk is in alternative dQ parallelization strategies, which are evaluated and benchmarked explicitly in Section 4.F.

Principle 3 — Recompute Rather Than Rematerialize

The backward pass deliberately re-executes the forward pass tile computations rather than storing the $N \times N$ attention matrix for later use. This trades approximately $2\times$ additional FLOPs for complete elimination of $O(N^2)$ HBM storage between the forward and backward passes. For memory-bandwidth-bound workloads on modern GPUs — where compute is cheap and memory bandwidth is the bottleneck — this trade-off is strongly favourable. The recomputation cost is quantified explicitly in the benchmarking analysis.

4. Baselines and Benchmarking Configuration

Baseline	Implementation	Purpose
Naive PyTorch	<code>torch.nn.functional.scaled_dot_product_attention</code> with <code>enable_flash=False, enable_math=True</code>	Worst-case $O(N^2)$ memory and speed for both passes
Custom Triton FlashAttention	This project's forward + backward implementation	Primary deliverable
Official FlashAttention-2	<code>flash-attn</code> package v2.x	Production upper bound
PyTorch FlexAttention	<code>torch.nn.attention.flex_attention</code>	Compiler-driven modern baseline

Test Configuration:

- Hardware: NVIDIA RTX 3090 (24 GB VRAM, 35.6 TFLOP/s FP32, 99 KB shared memory per SM) (Better GPU will be optional based on school provided); hardware locked in and documented at project start
- Sequence lengths: $N \in \{1024, 2048, 4096, 8192, 16384\}$
- Head dimension: $d = 64$
- Batch size: $B = 1$ for memory scaling isolation; $B = 8$ for throughput benchmarking
- Number of heads: $H = 8$
- Data type: FP32 for correctness verification; BF16 for performance benchmarking, FP16 for secondary benchmarking
- Warm-up protocol: 10 warm-up iterations discarded; 100 timed iterations averaged; `torch.cuda.synchronize()` called before each timing measurement
- Memory measurement: `torch.cuda.reset_peak_memory_stats()` before each run; `torch.cuda.max_memory_allocated()` after

5. Performance Evaluation

A. Numerical Correctness — Forward Pass

- `torch.allclose(custom_output, naive_output, rtol=1e-3, atol=1e-3)` across all sequence lengths and data types
- Maximum absolute element-wise deviation logged to detect systematic divergence

B. Numerical Correctness — Backward Pass

- `torch.autograd.gradcheck(flash_attention_fn, inputs, eps=1e-4, rtol=1e-3, atol=1e-3)` on small inputs ($N = 64$, $d = 32$) — finite difference verification of all three gradients
- Direct gradient tensor comparison `torch.allclose(dQ_custom, dQ_naive, rtol=1e-3, atol=1e-3)` at full sequence lengths
- Separate validation of dQ , dK , and dV independently to isolate any per-gradient errors

C. End-to-End Training Validation

- A small GPT-2 style single-transformer-block model will be trained for 100 iterations using the custom FlashAttention as the attention backend
- Loss curve will be compared against identical training using naive PyTorch attention to confirm that gradient correctness translates to equivalent optimization behavior

D. Execution Speed

- Wall-clock time (milliseconds) and effective throughput (TFLOP/s) measured separately for forward pass and backward pass
- Both passes measured across all four baselines on identical hardware

E. Memory Footprint

- Peak VRAM measured separately for forward pass, backward pass, and combined forward + backward (training scenario)
- Expected: Naive PyTorch retains $N \times N$ matrix between passes during training, roughly doubling VRAM at peak; custom implementation maintains $O(N)$ throughout

F. Implementation Complexity Analysis

Metric	Naive PyTorch	Custom Triton	FlashAttention-2	FlexAttention
Lines of code (core logic)	TBD	TBD	TBD	TBD
HBM read/write operations	$O(N^2)$	$O(N)$	$O(N)$	TBD
Kernel launches (fwd + bwd)	TBD	TBD	TBD	TBD
Custom masking support	Limited	Yes (causal)	Limited	Yes (flexible)
Training capable	Yes	Yes	Yes	Yes

6. Development Components, Debugging Strategy, and Bottleneck Mitigation

Component A: Pure Python/PyTorch Mathematical Prototype

Before writing any GPU code, both the forward and backward pass algorithms will be implemented as explicit nested Python loops operating on standard PyTorch CPU tensors. This prototype will be validated against `torch.nn.functional.attention` (forward) and `torch.autograd.grad` (backward).

Forward Prototype Unit Tests:

- Correctness on single-head, single-batch inputs at $N = 64$
- Correctness with causal mask enabled and disabled
- Numerical stability with extreme input values

Backward Prototype Unit Tests:

- D_i precomputation matches $\text{rowsum}(dO \odot O)$ computed by autograd

- dV gradient matches finite difference estimate at $N = 64$
- dQ and dK gradients match finite difference estimates at $N = 64$
- Full gradcheck passes end-to-end

Component B: Triton Forward Kernel Development

Debugging Strategy — Forward Kernel:

1. Implement for $B_r = B_c = 1$ (degenerates to standard attention) and verify at every step
2. Use `tl.store()` to write intermediate values (S_{ij} , m_{ij} , correction factors α) to debug buffers readable from Python
3. Progressively expand block sizes ($1 \rightarrow 16 \rightarrow 32 \rightarrow 64$) verifying correctness at each step
4. Apply `triton.autotune()` to sweep (B_r, B_c) candidates on target hardware

Component C: Triton Backward Kernel Development

Debugging Strategy — Backward Kernel:

1. Implement and validate D_i precomputation kernel first in isolation — verify against $\text{rowsum}(dO \odot O)$ from PyTorch autograd
2. Implement dV computation first — it has no atomic addition issue and is the simplest gradient to verify
3. Implement dK computation second — also no atomic issue when the outer loop is over column blocks
4. Implement dQ computation last — this requires the atomic addition or sequential loop strategy
5. Run gradcheck after each gradient is added, isolating any error to the most recently added component
6. Compare full (dQ, dK, dV) against naive PyTorch backward at $N = 128$ before scaling up

Component D: Benchmark Harness and Training Validation

A unified Python benchmarking script will call all four implementations through a common interface. A separate training validation script will run the small GPT-2 block training test.

Bottleneck Mitigation

Bottleneck	Description	Mitigation
Shared Memory Overflow	Block size too large causes SRAM overflow	Query device shared memory at runtime; compute maximum Br, Bc analytically before kernel launch
dQ Atomic Contention	Atomic additions for dQ create memory bottleneck	Evaluate sequential j-loop strategy first; benchmark atomic vs. sequential trade-off explicitly
Backward Gradient Bug	Subtle incorrect gradient silent at large N	Implement gradcheck at small N; validate each of dQ, dK, dV independently before combining
Hardware Incompatibility	Triton version mismatches cause silent errors	Lock CUDA 12.x, Triton 2.x, PyTorch 2.x in requirements.txt; verify on target GPU at project start
Timeline Slippage	Backward kernel debugging overruns into thesis writing	Explicit gate: backward pass dropped if forward pass not complete by end of December 2026
gradcheck Slowness	Finite difference check is prohibitively slow at large N	Run gradcheck only at $N \leq 128$; use direct tensor comparison for larger sequences

D. Timeline

Period	Phase	Milestone	Deliverable
June 2026	Setup	Project statement submission; supervisor gate condition agreement; GPU environment identification; CUDA/Triton version lockdown	Approved project statement
July 2026	Literature	Full reading of FlashAttention-1, FlashAttention-2, Sparse Attention, Longformer, Performer papers; backward pass mathematical derivation study	Annotated reference list; handwritten derivation of dQ, dK, dV
August 2026	Prototype	Component A: Python/PyTorch forward AND backward prototype; unit tests for all three gradients; gradcheck validation	Validated Python prototype — forward and backward — with full test suite
September 2026	Triton Fwd	Component B: Triton environment setup; scalar-first forward kernel; progressive block size expansion; causal masking integration	Initial functional Triton forward kernel passing correctness checks
October 2026	Triton Fwd	(2-week contingency buffer) Forward kernel stabilized; autotuning sweep; forward benchmark harness wired up	1st Progress Report submitted; forward pass benchmarks complete
November 2026	Triton Bwd	Gate Check: Forward pass fully benchmarked — proceed to backward pass. Component C begin: D_i precomputation kernel; dV kernel; dK kernel	1st incremental source code (forward kernel + prototype + partial benchmark harness)
December 2026	Triton Bwd	dQ kernel with atomic/sequential strategy; torch.autograd.Function integration; gradcheck validation; full	2nd incremental source code (complete forward +

		forward + backward correctness confirmed	backward kernel + benchmark harness)
January 2027	Benchmarking	Core profiling: forward + backward speed and VRAM across all four baselines and all sequence lengths; GPT-2 block training validation; hyperparameter sensitivity analysis	2nd Progress Report + Mid-Point Presentation and Live System Demonstration
February 2027	Analysis	Data aggregation; ablation studies on block sizes; scaling law curve fitting; recomputation FLOP cost analysis; thesis writing begins	4th monthly source code iteration; thesis draft Sections A–C complete
March 2027	Writing	Results chapter; discussion including recomputation trade-off analysis; complexity comparison table; conclusion; supervisor review	Final archived source code; complete thesis draft submitted for review
April 2027	Submission	Thesis final revisions; softcopy submission to BUMoodle; oral defense preparation and delivery; asset archiving	Final PDF; poster; demo video; setup ZIP; oral defense

Gate Condition (Wait for approval)

If the Triton forward pass kernel is not fully implemented, benchmarked against all four baselines, and correctness-validated by the end of November 2026, the backward pass implementation will be dropped from the project scope. In this scenario, the backward pass will be documented in the thesis with complete mathematical derivations, pseudocode, identified Triton implementation challenges, and a detailed plan for future implementation. This gate protects thesis writing time and ensures a complete, high-quality forward pass analysis is delivered regardless of backward pass progress.

E. Future Work

AMD ROCm Port

The validated NVIDIA Triton kernels will serve as the reference implementation for a future cross-platform study targeting AMD GPU hardware. Key open questions identified for future investigation:

- Wavefront size impact: AMD executes threads in wavefronts of 64 (vs. NVIDIA's warps of 32). Optimal block sizes B_r and B_c will differ, and warp-level reduction operations in the online softmax must be redesigned for 64-thread wavefronts
- LDS vs. SRAM behavioral differences: AMD's Local Data Share has different bank conflict patterns due to wavefront size; the block size selection formula must be revalidated against empirical LDS utilization rather than taken from documentation alone
- HBM/SRAM bandwidth ratio on AMD hardware: The IO-aware efficiency claims of FlashAttention assume a specific ratio of HBM to SRAM bandwidth. Whether these claims transfer proportionally to AMD's MI250/MI300X or consumer RX 7000 series is an open empirical question
- Profiling toolchain: Mapping NVIDIA Nsight Compute's HBM read/write byte counters to AMD's ROCm Profiler / Omniperm equivalents is non-trivial and is identified as a significant effort item for future work

FlashAttention-3 Features

Future work will explore integrating Hopper-architecture-specific features into the Triton kernel:

- FP8 quantization: Implementing attention computation in 8-bit floating point to further reduce memory bandwidth consumption
- Asynchronous memory pipelines: Overlapping SRAM loads with compute operations to hide memory latency

References

Beltagy, I., Peters, M. E., & Cohan, A. (2020). Longformer: The long-document transformer.

arXiv preprint arXiv:2004.05150.

Child, R., Gray, S., Radford, A., & Sutskever, I. (2019). Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509.*

Choromanski, K., Likhoshesterov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., & Wierstra, D. (2021). Rethinking attention with Performers. *International Conference on Learning Representations (ICLR 2021).*

Dao, T. (2023). FlashAttention-2: Faster attention with better parallelism and work partitioning. *International Conference on Learning Representations (ICLR 2023).*

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *Advances in Neural Information Processing Systems (NeurIPS 2022).*

Milakov, M., & Gimelshein, N. (2018). Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867.*

Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., & Dao, T. (2024). FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. *arXiv preprint arXiv:2407.08608.*

Wang, S., Li, B., Khabsa, M., Fang, H., & Ma, H. (2020). Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768.*